I0 · Importer des fichiers réels

~2.5 h

À la fin de cette séance vous saurez

- lire un fichier **Excel** à plusieurs feuilles (`readxl`), sauter des lignes de titre et choisir la feuille ;
- vérifier ce que R a importé : types, dimensions, valeurs manquantes, valeurs étranges ;
- gérer les **accents et encodages**, les virgules décimales et les séparateurs `;` ;
- produire une première version propre des données et l'enregistrer.

Le fil rouge du niveau intermédiaire

Toutes les séances de ce niveau suivent la même organisation (simulée) : la coopérative de producteurs d'algues **Fanantenana**, dans 12 villages d'une baie fictive. La coopérative vous demande de répondre à des questions comme : *Quels villages et quelles saisons donnent la meilleure croissance ? La croissance est-elle liée à la température de l'eau ? Les femmes et les hommes utilisent-ils la même technique ?* Chaque séance vous rapproche des réponses.

Fichier	Contenu	Utilisé en
`seaweed_farming.xlsx`	producteurs et lignes de culture (2 ans)	I0, I1, I3, I5–I8, I10, I11
`water_quality_long.csv` , `water_quality_wide.csv` , `stations.csv`	mesures à la sonde sur 10 stations	I2, I3, I7, I8, I10, I11
`survey_households.csv` , `survey_members.csv` , `survey_assets.csv`	enquête ménages dans 6 villages	I3, I4, I9, I10
`villages.csv` , `coastline.csv`	coordonnées pour les cartes	I3, I11
`seacucumber_*.csv`	enclos d'holothuries (projet final)	I13

⇩ r_course_data_intermediate.zip **⇩ check_setup.R**

i **Avant de commencer :** créez un nouveau projet RStudio `R_course_intermediate` avec les dossiers `data/`, `scripts/`, `outputs/` ; décompressez les données dans `data/` ; installez les packages de ce niveau et lancez `check_setup.R` avec `LEVEL <- "intermediate"` :

```
install.packages(c("readxl", "tidyr", "lubridate", "stringr", "patchwork", "dplyr", "ggplot2"))
```

Comment les données d'algues ont été collectées

~ **Contexte simulé.** De janvier 2024 à décembre 2025, le technicien de la coopérative a suivi les 96 producteurs membres. L'algue (*Kappaphycus alvarezii*) est cultivée sur des **lignes** : les boutures sont attachées sur une corde fixée au-dessus du fond (*off-bottom*) ou flottante (*long-line*). Le jour du semis, le technicien pèse les boutures de chaque ligne avec un peson (kg). Environ 45 jours plus tard, il pèse la récolte de la même ligne. Les lignes perdues lors de tempêtes ou broutées par les poissons n'ont pas de poids final. Il saisit tout dans un classeur Excel : une feuille pour les producteurs, une pour les lignes, et une feuille README qui décrit les données.

Type de données : un registre de mesures de production répétées (quantitatives : poids, longueurs, dates ; qualitatives : village, sexe, technique, statut).

Lire des fichiers Excel avec readxl

`read.csv()` ne lit pas les fichiers `.xlsx`. Le package **readxl** (de la famille tidyverse, Excel n'est pas nécessaire) le fait. Listez d'abord les feuilles, puis lisez celle dont vous avez besoin.

```
library(readxl)
excel_sheets("data/seaweed_farming.xlsx")              # names of the sheets
```

```
#> [1] "README"  "farmers" "lines"
```

```
read_excel("data/seaweed_farming.xlsx", sheet = "README", col_names = FALSE)
```

```
#> # A tibble: 6 × 1
#>   ...1
#>   <chr>
#> 1 Fanantenana seaweed cooperative — farming register (SIMULATED DATA for teach
i…
#> 2 Collected by the cooperative technician, Jan 2024 - Dec 2025.
#> 3 Each line of cuttings (Kappaphycus) is weighed with a hanging scale at plant
i…
#> 4 Sheet 'farmers': one row per farmer. Sheet 'lines': one row per farming line.
#> 5 Weights in kg. harvest_date was typed by hand as dd/mm/yyyy.
#> 6 status: harvested / lost - storm / grazed by fish (no final weight when the
l…
```

```
farmers <- read_excel("data/seaweed_farming.xlsx", sheet = "farmers")
farmers
```

```
#> # A tibble: 96 × 7
#>    farmer_id village  sex     age experience_years joined_coop         techniqu
e
#>    <chr>     <chr>    <chr> <dbl>            <dbl> <dttm>              <chr>
#>  1 F001      Sarondr… F        34                1 2022-04-21 00:00:00 off-bot
t…
#>  2 F002      Ambatof… F        33                1 2022-10-03 00:00:00 long-lin
e
#>  3 F003      Andrevo… F        43                2 2022-02-22 00:00:00 off-bot
t…
#>  4 F004      Tsimanj… F        50               17 2021-01-24 00:00:00 long-lin
e
#>  5 F005      Ambalak… M        60                8 2019-05-18 00:00:00 off-bot
t…
#>  6 F006      Befasy   M        19                3 2021-07-02 00:00:00 off-bot
t…
#>  7 F007      Ampasim… F        31                2 2021-10-29 00:00:00 long-lin
e
#>  8 F008      Mahavelo F        54                3 2022-12-09 00:00:00 off-bot
t…
#>  9 F009      MAHAVELO F        31                0 2019-07-30 00:00:00 long-lin
e
#> 10 F010      Antsira… M        21                5 2023-09-11 00:00:00 off-bot
t…
#> # ℹ 86 more rows
```

Le résultat est un **tibble** : un data frame moderne qui n'affiche que les premières lignes et indique le type de chaque colonne sous son nom (`<chr>` texte, `<dbl>` nombre, `<dttm>` date-heure). Tout ce que vous savez sur les data frames fonctionne sur les tibbles. Pour obtenir un data frame classique : `as.data.frame()` .

Feuilles avec des lignes de titre

Ouvrez la feuille `lines` dans Excel : le technicien a écrit deux lignes de titre au-dessus du tableau, puis une ligne vide. Lue naïvement, R prend le titre comme nom de colonne :

```
bad <- read_excel("data/seaweed_farming.xlsx", sheet = "lines")
names(bad)[1:3]
```

```
#> [1] "Seaweed lines register — Fanantenana cooperative"
#> [2] "...2"
#> [3] "...3"
```

`skip = 3` ignore les 3 premières lignes. Autres arguments utiles : `range = "A4:J200"` (un bloc exact de cellules), `na = c("", "NA", "-")` (valeurs à lire comme manquantes), `col_types` (imposer les types).

```
lines <- read_excel("data/seaweed_farming.xlsx", sheet = "lines", skip = 3)
dim(lines)
```

```
#> [1] 2575   10
```

```
head(lines, 3)
```

```
#> # A tibble: 3 × 10
#>   line_id farmer_id cycle planting_date       harvest_date line_length_m
#>   <chr>   <chr>     <dbl> <dttm>              <chr>                <dbl>
#> 1 L0001   F013          1 2024-01-08 00:00:00 22/02/2024              15
#> 2 L0002   F013          1 2024-01-08 00:00:00 20/02/2024              15
#> 3 L0003   F013          1 2024-01-08 00:00:00 26/02/2024              15
#> # i 4 more variables: initial_weight_kg <dbl>, final_weight_kg <dbl>,
#> #   status <chr>, notes <chr>
```

Vérifier ce qui a été importé

Un vrai fichier cache toujours des surprises. Faites de cette vérification une routine à **chaque** import : dimensions, types, valeurs manquantes, étendues, catégories.

```
str(lines)
```

```
#> tibble [2,575 × 10] (S3: tbl_df/tbl/data.frame)
#>  $ line_id          : chr [1:2575] "L0001" "L0002" "L0003" "L0004" ...
#>  $ farmer_id        : chr [1:2575] "F013" "F013" "F013" "F013" ...
#>  $ cycle            : num [1:2575] 1 1 1 1 1 1 1 1 1 1 ...
#>  $ planting_date    : POSIXct[1:2575], format: "2024-01-08" "2024-01-08" ...
#>  $ harvest_date     : chr [1:2575] "22/02/2024" "20/02/2024" "26/02/2024" "18/0
2/2024" ...
#>  $ line_length_m    : num [1:2575] 15 15 15 15 20 20 20 20 20 20 ...
#>  $ initial_weight_kg: num [1:2575] 8.4 10.2 8.2 10.2 11.2 10.4 12.5 13.5 10.2
8.2 ...
#>  $ final_weight_kg  : num [1:2575] 55.3 47.8 51.1 NA 35.1 44.5 40.5 57.3 41.1 3
9.7 ...
#>  $ status           : chr [1:2575] "harvested" "harvested" "harvested" "grazed
by fish" ...
#>  $ notes            : chr [1:2575] NA NA NA NA ...
```

```
colSums(is.na(lines))            # missing values per column
```

```
#>           line_id         farmer_id             cycle     planting_date
#>                 0                 0                 0                 0
#>      harvest_date     line_length_m initial_weight_kg   final_weight_kg
#>                 0                 0                 0               258
#>            status             notes
#>                 0              2402
```

```
summary(lines$final_weight_kg)   # look at the maximum!
```

```
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
#>    15.8    39.9    50.7   226.4    65.4 92600.0     258
```

```
table(lines$status)
```

```
#>
#> grazed by fish      harvested   lost - storm
#>            115           2317            143
```

Trois choses à noter :

1. `planting_date` est une **date-heure** (`POSIXct`) car c'était une vraie date dans Excel, alors que `harvest_date` est du **texte** : elle a été saisie à la main en `jj/mm/aaaa` . Nous corrigerons les deux en séance I1.
2. `final_weight_kg` a 258 `NA` — les lignes perdues — et un maximum de plusieurs milliers : certains poids ont été saisis en **grammes**.
3. Les noms de villages dans `farmers` ne sont pas cohérents.

```r
sort(unique(farmers$village))
```

```
#>  [1] "Ambalakely"  "AMBALAKELY"  "Ambatofotsy" "Ampasimena"  "Andrevobe"
#>  [6] "ANDREVOBE"   "Ankarabe"    "Antsiraka"   "Befasy"      "BEFASY"
#> [11] "Lovobe"      "LOVOBE"      "Mahavelo"    "MAHAVELO"    "Sarondrano"
#> [16] "Tsiandavaka" "Tsimanjaka"  "TSIMANJAKA"
```

```r
farmers$village <- tools::toTitleCase(tolower(trimws(farmers$village)))   # Beginner, Session 5
length(unique(farmers$village))
```

```
#> [1] 12
```

> **À vous.** Combien de lignes ont un poids final supérieur à 200 kg ? Quel producteur a le plus de lignes ? (`table()` puis `sort()`).

`sum(lines$final_weight_kg > 200, na.rm = TRUE)` — et `sort(table(lines$farmer_id), decreasing = TRUE)[1]` .

Fichiers CSV : séparateurs, décimales, accents

Problème	Symptôme	Solution
Séparateur `;` (Excel français)	toutes les données dans une colonne	`read.csv2()` ou `sep = ";"`
Virgule décimale `12,5`	nombres lus comme du texte	`dec = ","`
Accents `é` affichés `Ã©`	mauvais encodage	`fileEncoding = "UTF-8"` ou `"latin1"`
Manquants écrits `-` , `.` , `NA` , `999`	nombres lus comme texte, ou fausses valeurs	`na.strings = c("", "-", ".")` , puis corriger les codes comme `-999`
Lignes vides à la fin	lignes pleines de NA	les retirer avec `df[rowSums(!is.na(df)) > 0, ]`

```r
wq <- read.csv("data/water_quality_long.csv")
str(wq)
```

```
#> 'data.frame':    720 obs. of  7 variables:
#>  $ station      : chr  "WQ01" "WQ02" "WQ03" "WQ04" ...
#>  $ date         : chr  "05/01/2023" "05/01/2023" "05/01/2023" "05/01/2023" ...
#>  $ temp_C       : num  28.7 29 28.6 28.2 28 27.8 27.5 27.7 28.2 28.4 ...
#>  $ salinity_psu : num  33.3 33.1 33.4 33.2 33.3 34 35 33.1 32.6 32.2 ...
#>  $ pH           : num  8.17 8.16 8.08 8.15 8.12 8.18 8.08 8.08 8.03 8.1 ...
#>  $ DO_mgL       : num  7.24 6.57 6.46 6.64 6.75 6.71 6.55 7.02 7.35 6.35 ...
#>  $ turbidity_NTU: num  2.8 2.5 2.8 5.2 3.7 2.1 4.5 11.4 6.7 8 ...
```

```
range(wq$temp_C, na.rm = TRUE)     # -999 is a probe "error code", not a temperatu
re
```

```
#> [1] -999   30
```

`readr::read_csv()` (tidyverse) est une alternative plus rapide qui renvoie un tibble et devine les types plus prudemment. Les deux conviennent : utilisez ce que vous préférez, mais **vérifiez toujours** le résultat.

Enregistrer une copie propre

Ne modifiez jamais le fichier brut. Enregistrez la version nettoyée dans `outputs/` (ou `data/clean/`) avec un nom qui dit ce qu'elle contient. La séance suivante peut partir du fichier de contrôle ci-dessous si vous voulez être sûr de partir de données correctes.

```
lines$planting_date <- as.Date(lines$planting_date)    # date-time -> date
write.csv(farmers, "outputs/I0_farmers.csv", row.names = FALSE)
write.csv(lines,   "outputs/I0_lines.csv",   row.names = FALSE)
```

⇩ checkpoint: I0_farmers.csv **⇩ checkpoint: I0_lines.csv**

Travail d'application et quiz

Travail d'application I0

Nouveau script `scripts/i00_import.R`.

1. Combien de feuilles contient `seaweed_farming.xlsx` ?
2. Lisez correctement la feuille `lines`. Combien de lignes et de colonnes ?
3. Combien de lignes n'ont pas de poids final (`NA`) ?
4. Combien de poids finaux dépassent 200 (saisis en grammes) ?
5. Lisez la feuille `farmers`. Combien d'orthographes de villages différentes avant nettoyage ? Et après ?
6. Quel village compte le plus de producteurs (après nettoyage) ?

by Aroo Falinirina

7. Combien de producteurs sont des femmes (`sex == "F"`) ?
8. Lisez `water_quality_long.csv` . Combien de températures sont codées `-999` ?

> Pour la question 2, pensez aux lignes de titre : `skip = 3` . Pour la 5, comptez avec `length(unique(...))` avant et après `tools::toTitleCase(tolower(trimws(...)))` .

Quiz : répondez en ligne sur le site du cours (votre score y est enregistré).

by Aroo Falinirina